

COMPUTACIÓN EN INTERNET 2

Guía Completa de Estudio

Protocolos · Servidores · Tomcat · Servlets · Spring Framework

15 Clases · Explicaciones con metáforas · Preguntas de repaso

Universidad ICESI · 2026

CLASE 1

TCP y UDP

La base de toda comunicación en red

¿Qué es un protocolo?

Un **protocolo** es un conjunto de reglas que dos partes acuerdan seguir para poder comunicarse. Sin reglas comunes, la comunicación es imposible.

■ Metáfora

Cuando llamas a alguien por teléfono hay un protocolo implícito: esperas que contesten, dices 'hola', la otra persona responde. Si alguien ignorara esas reglas y hablara sin esperar, nadie se entendería. TCP y UDP son exactamente eso — reglas acordadas para mandar datos entre computadores.

Protocolo TCP — El mensajero responsable

TCP (*Transmission Control Protocol*) garantiza que los datos llegan **completos, en orden y sin errores**. Su prioridad es la **confiabilidad**, no la velocidad.

■ Metáfora

Imagina que mandas 10 cajas numeradas por mensajería premium. Si la caja #3 se pierde, el mensajero vuelve a buscarla — no entrega el resto hasta que lleguen todas en orden. Si llegan desordenadas, las reordena antes de dártelas.

El Three-Way Handshake — El saludo de 3 pasos

Antes de mandar datos, TCP establece una conexión formal con 3 mensajes:

```
Cliente Servidor | | 1. SYN —————> | 'Quiero conectarme' | | <— 2. SYN-ACK
————— | 'Recibido, yo también quiero' | | 3. ACK —————> | 'Confirmado,
empecemos' | | CONEXIÓN ESTABLECIDA |
```

Paso	Emisor	Mensaje	Significado
1	Cliente → Servidor	SYN	¿Podemos conectarnos?
2	Servidor → Cliente	SYN-ACK	Sí, te escucho, ¿me escuchas tú?
3	Cliente → Servidor	ACK	Confirmado, todo listo

Ventana deslizante (Sliding Window)

TCP no espera un ACK por cada byte — envía bloques completos según la capacidad del receptor. El receptor notifica cuántos bytes puede recibir (*Window Size*). Si un paquete no recibe confirmación en el tiempo límite, TCP lo retransmite automáticamente.

■ Metáfora

Es como cargar un camión de mudanzas. No llevas una silla, vuelves por otra y así. Cargas todo lo que cabe en el camión (la ventana), lo entregas, confirmas que llegó, y vuelves por más.

Cierre de conexión — El adiós de 4 pasos

TCP cierra la conexión con **4 mensajes** porque cada dirección se cierra de forma independiente:

```
1. FIN (Cliente → Servidor) 'Ya terminé de enviar' 2. ACK (Servidor → Cliente) 'Recibí que terminaste' 3. FIN (Servidor → Cliente) 'Yo también terminé' 4. ACK (Cliente → Servidor) 'Recibido, conexión cerrada'
```

Son 4 y no 3 porque cuando el cliente dice 'termine' (FIN), el servidor puede aun tener datos por mandar. El ACK confirma el FIN del cliente, pero la conexión del servidor se cierra después con su propio FIN.

Protocolo UDP — El mensajero relajado

UDP (*User Datagram Protocol*) es el polo opuesto. No establece conexión, no verifica si los datos llegaron, no pide confirmación. Manda el dato y sigue de largo. Su cabecera tiene solo **8 bytes** vs los **20 bytes** de TCP — menos control = más velocidad.

■ Metáfora

Es como gritarle algo a un amigo en un concierto lleno de gente. Si el ruido hace que se pierda una palabra, no te devuelves a repetirla — sigues hablando. Rápido, pero con riesgo de perder información.

UDP se usa donde **la velocidad importa más que la perfección**:

- ■ Juegos en línea — un dato de posición perdido no detiene el juego
- ■ Videollamadas — un frame perdido es un glitch de un segundo, mejor que congelar
- ■ Streaming — mismo caso
- ■ DNS — consultas rapidísimas donde la velocidad manda

Comparación TCP vs UDP

Característica	TCP	UDP
Conexión previa	Sí (handshake 3 vías)	No
Garantía de entrega	Sí	No
Orden de datos	Garantizado	No garantizado
Retransmisión	Sí, automática	No
Velocidad	Más lento	Más rápido
Tamaño cabecera	20 bytes	8 bytes
Casos de uso	Web, correo, archivos, SSH	Streaming, DNS, juegos, VoIP

■ Preguntas de repaso

1. Si ves fútbol en streaming y la imagen se congela un segundo pero sigue, ¿qué protocolo se usa y por qué ese comportamiento es aceptable?
2. El cierre de TCP necesita 4 pasos y el inicio solo 3. ¿Por qué el cierre necesita uno más? Explícalo con tus palabras.
3. ¿Qué pasaría si TCP no tuviera ventana deslizante y esperara un ACK por cada byte? ¿Qué problema generaría?
4. Alguien dice 'UDP es malo porque pierde datos'. ¿Estás de acuerdo? Justifica.

CLASE 2

Cliente-Servidor, Puertos, Sockets y HTTP

El modelo mental antes del código

¿Qué es un servidor web?

Un servidor web es **un programa** que espera conexiones en un puerto y, cada vez que alguien le pide un recurso, se lo devuelve. No es una máquina, no es un edificio: es un proceso corriendo, y puede estar corriendo en tu propio portátil.

Término	Qué es
Servidor web	El programa que atiende peticiones HTTP
Servidor (hardware)	La máquina donde ese programa corre
Servidor de aplicaciones	Programa que además ejecuta lógica para construir la respuesta (ej: Tomcat)

Un servidor web hace **4 cosas en ciclo, para siempre**: escucha en un puerto → acepta una conexión → lee la petición → escribe la respuesta y cierra.

El modelo cliente-servidor

La comunicación **siempre la inicia el cliente**. El servidor nunca llama al navegador — solo espera. Esta asimetría define todo el protocolo HTTP.

■ Cuando el navegador recibe un HTML que referencia 5 imágenes, repite el ciclo completo 5 veces más — una conexión por cada imagen. Una página con 15 recursos son 16 peticiones. Por eso el servidor debe atender varias a la vez.

Puertos — el número del apartamento

Una máquina tiene **una sola dirección IP** pero muchos programas queriendo hablar por red. El **puerto** es el número que dice a cuál de todos esos programas va dirigido el tráfico.

■ Metáfora

La IP es como la dirección de un edificio de apartamentos. El puerto es el número del apartamento. Todos los paquetes llegan al mismo edificio (misma IP), pero el número del apartamento (puerto) decide quién los recibe.

- Puertos 0–1023: reservados para servicios estándar. HTTP usa el 80, HTTPS el 443.
- Para las prácticas se usa un puerto libre por encima de 1024 — por ejemplo **6789**. Ese número va también en la URL: `http://localhost:6789/index.html`

Sockets — el tubo bidireccional

Un **socket** es la abstracción con la que el sistema operativo te deja usar una conexión de red como si fuera un archivo: tiene un stream de entrada (lees) y uno de salida (escribes).

■ Metáfora

El socket es un tubo bidireccional entre tu programa y el cliente. Por un lado entra lo que el cliente manda, por el otro sale lo que tú respondes. Tu programa no tiene que saber nada de cables ni señales.

Socket	Cuántos	Para qué sirve
ServerSocket (escucha)	Uno, toda la ejecución	Solo acepta conexiones. No transporta datos.
Socket (conexión)	Uno por cliente, nace y muere por petición	Por aquí sí viajan los bytes.

```
// El de escucha – uno, siempre abierto ServerSocket serverSocket = new ServerSocket(6789); // El
de conexión – uno nuevo por cada cliente Socket connectionSocket = serverSocket.accept(); //
bloquea hasta que llega alguien
```

Anatomía de una petición HTTP

HTTP es un protocolo de **texto plano**. Lo que el navegador manda por el socket es literalmente esto:

```
GET /index.html HTTP/1.0 Host: localhost:6789 User-Agent: Mozilla/5.0 Accept: text/html <- línea
vacía OBLIGATORIA
```

Parte	Ejemplo	Qué es
Línea de solicitud	GET /index.html HTTP/1.0	Método + recurso + versión del protocolo
Headers	Host: localhost:6789	Pares Nombre:valor, uno por línea
Línea en blanco	(vacía)	Señal de 'ya terminé de hablar'
Body	(opcional en GET)	Datos enviados en POST

Anatomía de una respuesta HTTP

```
HTTP/1.0 200 OK Content-Type: text/html Content-Length: 43 <html><body><h1>It
works!</h1></body></html>
```

Código	Significado
200 OK	Todo bien, aquí está lo que pediste
404 Not Found	El recurso no existe
500 Internal Server Error	El servidor se rompió

CRLF — el detalle que rompe todo

■ La especificación exige que cada línea del mensaje HTTP termine con `\r\n` (retorno de carro + salto de línea), NO con `\n` a secas. Es la causa número uno de servidores que 'no responden'. El navegador se queda esperando headers para siempre si falta la línea vacía final.

```
final static String CRLF = "\r\n";
```

■ Preguntas de repaso

1. ¿Cuál es la diferencia entre `ServerSocket` y `Socket` de conexión? ¿Por qué necesitas los dos?
2. Identifica el método, recurso y versión en: `GET /imagenes/foto.jpg HTTP/1.0`
3. El navegador se queda cargando para siempre sin error en consola. ¿Cuáles son las dos causas más probables?
4. ¿Por qué `Content-Type` es tan importante? ¿Qué pasa si se omite al enviar una imagen JPEG?

CLASE 3

Servidor de una sola petición

El ciclo completo de HTTP sin distracciones

¿Por qué empezar con algo 'incompleto'?

Un servidor real tiene dos complejidades encima: el bucle infinito y los hilos. Si las metes desde el principio, cuando algo falla no sabes si el problema es HTTP o concurrencia. Aquí hay **un cliente, una petición, una respuesta, el programa termina**.

El código completo

```
import java.io.*; import java.net.*; import java.nio.charset.StandardCharsets; public class
SimpleWebServer { public static void main(String[] args) throws Exception { final String CRLF =
"\r\n"; int port = 6789; // PASO 1: Socket de escucha ServerSocket serverSocket = new
ServerSocket(port); // PASO 2: Espera bloqueante Socket connectionSocket = serverSocket.accept();
// PASO 3: Streams BufferedReader in = new BufferedReader( new
InputStreamReader(connectionSocket.getInputStream())); DataOutputStream out = new
DataOutputStream( connectionSocket.getOutputStream()); // PASO 4: Lee la petición String
requestLine = in.readLine(); String headerLine; while ((headerLine = in.readLine()) != null &&
!headerLine.isEmpty()) { System.out.println(headerLine); } // PASO 5: Construye y envía la
respuesta String body = "<html><body><h1>It works!</h1></body></html>"; byte[] bodyBytes =
body.getBytes(StandardCharsets.UTF_8); String head = "HTTP/1.0 200 OK" + CRLF + "Content-Type:
text/html; charset=utf-8" + CRLF + "Content-Length: " + bodyBytes.length + CRLF + CRLF;
out.write(head.getBytes(StandardCharsets.UTF_8)); out.write(bodyBytes); out.flush(); // PASO 6:
Cierra todo in.close(); out.close(); connectionSocket.close(); serverSocket.close(); } }
```

Detalles críticos a recordar

¿Por qué bodyBytes.length y no body.length()?

En UTF-8 una letra como **ó** ocupa 2 bytes pero es 1 carácter. Si usas `body.length()` (que cuenta caracteres), el número en Content-Length es incorrecto — el navegador espera más bytes de los que llegan y se queda cargando para siempre.

¿Por qué convertir el body a bytes ANTES que el head?

Porque necesitas `bodyBytes.length` para escribir el Content-Length en el head. No puedes escribir el peso en la etiqueta antes de haber pesado el paquete.

¿Por qué el while tiene dos condiciones?

```
while ((headerLine = in.readLine()) != null && !headerLine.isEmpty())
```

- `!= null` — si el cliente cierra abruptamente, `readLine()` devuelve null. Sin esta condicion, `isEmpty()` explotaría con `NullPointerException`.

- `!isEmpty()` — cuando `readLine()` lee la línea vacía que cierra los headers, devuelve "" (cadena vacía). Eso para el while.

¿Por qué flush() es obligatorio?

El `DataOutputStream` acumula bytes en un buffer interno. `flush()` los empuja de verdad hacia el socket. Sin él, los bytes se quedan atrapados en el buffer y el cliente nunca recibe nada — sin ningún error en consola.

■ Metáfora

`flush()` es como escribir una carta, meterla en un sobre y finalmente llevarla al buzón. Sin el flush la carta existe, está completa, pero nunca llegó al destinatario.

■ Preguntas de repaso

1. ¿Por qué el while de lectura de headers tiene dos condiciones (`!=null` y `!isEmpty()`)? ¿Qué pasaría si quitaras la condición `!=null`?
2. ¿Por qué se usa `bodyBytes.length` para el Content-Length en lugar de `body.length()`?
3. Headers y body se mandan en dos escrituras separadas aunque el body también sea texto. ¿Por qué se diseñó así?
4. Después de ejecutar el servidor y cargarlo en el navegador, si recargas dice 'connection refused'. ¿Es un error o el comportamiento esperado?

CLASE 4

Servidor Multi-hilos

Atender múltiples clientes en paralelo

¿Por qué un solo hilo no es suficiente?

Con un solo hilo, si el Cliente A está descargando un archivo grande que tarda 10 segundos, **todos los demás clientes esperan** esos 10 segundos. Una sola página puede generar 16 peticiones — con un hilo se atienden en fila.

■ Metáfora

Un restaurante con un solo mesero que atiende una mesa, va a la cocina, vuelve, entrega el plato, y solo entonces atiende la siguiente. Con múltiples meseros (hilos), todos atienden en paralelo sin bloquearse entre sí.

La estructura del servidor multi-hilos

Se divide en **dos clases**: `ServidorWeb` (el main con el bucle infinito) y `SolicitudHttp` (el manejador de cada petición, que corre en su propio hilo).

El método main — el bucle infinito

```
ServerSocket socketdeEscucha = new ServerSocket(6789); while (true) { // nunca termina Socket
socketdeConexion = socketdeEscucha.accept(); // espera cliente SolicitudHttp solicitud = new
SolicitudHttp(socketdeConexion); Thread hilo = new Thread(solicitud); hilo.start(); // lanza el
hilo y vuelve inmediatamente al while }
```

El `while(true)` tiene la condición fija `true` — nunca puede volverse `false`. El programa corre para siempre hasta que alguien lo mate con `Ctrl+C`.

`hilo.start()` no bloquea — lanza el hilo y regresa inmediatamente al `accept()` para esperar al siguiente cliente. `hilo.run()` ejecutaría el método directamente en el hilo principal (sin crear hilo nuevo), volviendo el servidor secuencial.

La clase `SolicitudHttp`

```
final class SolicitudHttp implements Runnable { final static String CRLF = "\r\n"; Socket socket;
public SolicitudHttp(Socket socket) throws Exception { this.socket = socket; // guarda el socket
de conexión } public void run() { // no puede lanzar excepciones try { proceseSolicitud(); } catch
(Exception e) { System.out.println(e); } } private void proceseSolicitud() throws Exception { //
aquí va toda la lógica de HTTP } }
```

- `final class` — la clase no puede ser heredada. Decisión de diseño.
- `implements Runnable` — obliga a definir `run()`, que es lo que ejecuta el hilo.
- `run()` no puede lanzar excepciones (contrato de `Runnable`). Por eso toda la lógica va en `proceseSolicitud()` que sí puede lanzarlas.
- `this.socket = socket` — guarda el socket del main en el atributo de clase para que `proceseSolicitud()` pueda usarlo.

Los métodos auxiliares

```
// Enviar el archivo en bloques de 1KB private static void enviarBytes(FileInputStream fis,
OutputStream os) throws Exception { byte[] buffer = new byte[1024]; int bytes = 0; while ((bytes =
fis.read(buffer)) != -1) { os.write(buffer, 0, bytes); // escribe exactamente los bytes leídos } }
// Determinar tipo MIME según extensión private static String contentType(String nombreArchivo) {
if (nombreArchivo.endsWith(".htm") || nombreArchivo.endsWith(".html")) return "text/html"; if
(nombreArchivo.endsWith(".gif")) return "image/gif"; if (nombreArchivo.endsWith(".jpg")) return
"image/jpeg"; return "application/octet-stream"; // tipo genérico – el navegador lo descarga }
```

¿Por qué buffer de 1KB y no leer todo de una vez? Porque un archivo de 500MB requeriría crear un arreglo de 500MB en RAM — el servidor se caería. Con el buffer de 1KB, sin importar el tamaño del archivo, siempre usas exactamente 1KB de memoria.

■■ os.write(buffer, 0, bytes) — el tercer argumento es el número exacto de bytes leídos en esta vuelta, no siempre 1024. El último bloque puede ser menor. Si pusieras siempre 1024 mandarías bytes de basura al final.

■ Preguntas de repaso

1. ¿Qué pasaría si usaras hilo.run() en lugar de hilo.start()? ¿Cuál es la diferencia?
2. ¿Por qué enviarBytes() usa un buffer de 1KB en lugar de leer todo el archivo de una vez?
3. contentType() devuelve 'application/octet-stream' cuando no reconoce la extensión. ¿Qué hace el navegador?
4. Un estudiante nota que a veces un cliente recibe el archivo corrupto. El buffer está declarado como atributo de la clase. ¿Cuál es la causa?

CLASE 5

Apache Tomcat y Java Servlets

Del servidor manual al contenedor profesional

¿Por qué no basta con el servidor de sockets?

Problema	Detalle
Parseo manual de HTTP	Manejar cookies, sesiones, query params a mano es complejo y propenso a errores
Gestión de hilos manual	Crear un hilo por petición sin control puede agotar la RAM
Acoplamiento	Código de sockets TCP mezclado con lógica de respuesta — imposible mantenerlo limpio
Sin estándar	Cada quien reinventa la rueda en seguridad y enrutamiento

¿Qué es Apache Tomcat?

Tomcat es un **contenedor de Servlets** (*Servlet Container*) de código abierto. En lugar de manejar sockets y bytes tú mismo, Tomcat lo hace y te entrega objetos Java listos para usar.

■ Metáfora

En la Clase 3 eras un cocinero que además tenía que construir la cocina, instalar los fogones y conectar el gas. Tomcat es la cocina ya construida y equipada — tú solo cocinas (escribes la lógica).

Coyote Connector: abre los sockets TCP, escucha en el puerto 8080, parsea las cabeceras HTTP y construye objetos Java con esa información.

Catalina Engine: administra las aplicaciones web desplegadas, mantiene un pool de hilos, y decide qué Servlet debe atender cada petición.

¿Qué es un Java Servlet?

Un **Servlet** es una clase Java que extiende `HttpServlet` y responde a peticiones HTTP dinámicas dentro de Tomcat.

```
@WebServlet("/hello") public class HolaServlet extends HttpServlet { @Override protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException { resp.setContentType("text/html"); resp.getWriter().println("<h1>Hola desde un Servlet</h1>"); } }
```

■ Con el Servlet no tienes que abrir `ServerSocket`, hacer `accept()`, crear streams, parsear la petición con `StringTokenizer`, construir headers manualmente ni hacer `flush()`. Tomcat se encarga de todo eso.

Ciclo de vida de un Servlet

Tomcat administra **3 etapas** estrictas. Crea **una sola instancia** del Servlet y la reutiliza para todas las peticiones. Por cada petición crea un **hilo nuevo** que ejecuta `service()` sobre esa única instancia.

Método	Cuántas veces	Para qué sirve
<code>init()</code>	Una sola vez al arrancar	Inicializar recursos (ej: obtener un bean de Spring)
<code>service()</code> → <code>doGet()/doPost()</code>	En cada petición	Lógica de respuesta según el método HTTP
<code>destroy()</code>	Una sola vez al apagar	Liberar recursos, cerrar conexiones

■ ■ Una sola instancia + múltiples hilos = los atributos de clase del Servlet son compartidos entre todos los hilos. Nunca uses atributos de clase para datos de una petición específica — usa variables locales dentro del método `doGet()/doPost()`.

HttpServletRequest y HttpServletResponse

```
// REQUEST – lo que el cliente mandó String nombre = req.getParameter("nombre"); // parámetro URL o formulario
String path = req.getRequestURI(); // /mi-app/estudiantes String agente = req.getHeader("User-Agent"); // qué navegador es
// RESPONSE – lo que vas a responder resp.setContentType("text/html; charset=UTF-8"); // tipo MIME
PrintWriter out = resp.getWriter(); // canal de escritura out.println("<h1>Hola</h1>"); // escribe el cuerpo
resp.sendRedirect("/otra-pagina"); // redirige con 302
```

`sendRedirect()` genera automáticamente un 302 Found con header Location. El navegador hace una nueva petición GET a esa URL. Se usa en el patrón Post-Redirect-Get para evitar que recargar la página reenvíe el POST.

■ Preguntas de repaso

1. Tomcat crea una sola instancia del Servlet y la reutiliza. ¿Qué implicación tiene eso para los atributos de clase?
2. ¿Cuál es la diferencia entre `init()` y `doGet()` en términos de cuántas veces se ejecutan?
3. ¿Por qué la dependencia `jakarta.servlet-api` tiene `scope='provided'` en el `pom.xml`?
4. En el servidor de sockets escribías 'HTTP/1.0 200 OK\r\n' manualmente. ¿Cómo haces lo equivalente en un Servlet?

CLASE 6

Spring Framework: SOLID, DIP e IoC

La base conceptual que sustenta Spring

¿Por qué necesitamos Spring?

Sin Spring, cada clase crea sus dependencias con `new`. Eso genera tres problemas:

Problema 1 — Alto acoplamiento: si cambias `EstudianteRepositoryInMemory` por `EstudianteRepositoryDatabase`, tienes que modificar todas las clases que la usan y recompilar todo.

Problema 2 — Imposible de probar: no puedes probar el servicio de forma aislada porque siempre crea su propio repositorio real adentro.

Problema 3 — Código repetitivo: en algún lado tienes que instanciar manualmente decenas de objetos — fontanería que no aporta valor al negocio.

La jerarquía: Principios → Patrones → Framework

Nivel	Qué es	Ejemplo
Principios (SOLID, IoC, DIP)	Filosofía de alto nivel — dice QUÉ buscar, 'Las clases deben estar desacopladas'	Las clases no deben estar desacopladas
Patrones (Factory, Strategy, DI)	Solución estructural probada — dice CÓMO estructurar las dependencias	Creación de las dependencias
Framework (Spring IoC)	Infraestructura ejecutable que automatiza Spring	Spring crea e inyecta beans automáticamente

■ Metáfora

Los principios son las leyes de tránsito (no conducir borracho, respetar señales). Los patrones son las técnicas de manejo (cómo hacer un paralelo). El framework es el carro con asistente de conducción — ya aplica muchas técnicas por ti.

Los principios SOLID

Letra	Nombre	Descripción
S	Single Responsibility	Una clase debe tener una sola razón para cambiar —
O	Open/Closed	Abierta para extensión, cerrada para modificación
L	Liskov Substitution	Una subclase puede reemplazar a su clase padre sin
I	Interface Segregation	Mejor varias interfaces específicas que una sola sobre
D	Dependency Inversion	Alto nivel no depende de bajo nivel — ambos depend

Dependency Inversion Principle (DIP)

Sin DIP — Alto acoplamiento

```
public class EstudianteServiceImpl { // depende DIRECTAMENTE de la clase concreta private
    EstudianteRepositoryInMemory repositorio; public EstudianteServiceImpl() { this.repositorio = new
    EstudianteRepositoryInMemory(); // acoplamiento fuerte } }
```

Con DIP — Bajo acoplamiento

```
public class EstudianteServiceImpl { // depende de la INTERFAZ, no de la implementación private
    EstudianteRepository repositorio; public EstudianteServiceImpl(EstudianteRepository repositorio) {
    this.repositorio = repositorio; // alguien externo decide la implementación } }
```

■ Metáfora

Un enchufe eléctrico (la interfaz) no le importa si lo que se conecta es una lámpara, un cargador o un ventilador (las implementaciones). El enchufe siempre es el mismo — lo que se conecta puede cambiar libremente.

Inversion of Control (IoC)

En la programación tradicional, **tú controlas** la creación de objetos. En IoC, ese control se **delega al Contenedor IoC de Spring**.

Sin IoC	Con IoC (Spring)
Tú construyes los objetos con new	Spring construye los objetos
Tú conectas las dependencias	Spring conecta las dependencias
Tú controlas el ciclo de vida	Spring gestiona el ciclo de vida

■ Metáfora

Sin IoC eres un cocinero que además va al mercado, lava y corta los ingredientes. Con IoC tienes un asistente (Spring) que te trae los ingredientes ya listos — tú solo cocinas (escribes la lógica de negocio).

■ Preguntas de repaso

1. ¿Cuál es la diferencia entre un principio, un patrón y un framework? Da un ejemplo de cada uno con Spring.
2. Explica qué viola este código y por qué: `private EstudianteRepositoryInMemory repo = new EstudianteRepositoryInMemory()`
3. ¿Cuál es la diferencia entre BeanFactory y ApplicationContext? ¿Cuál usarías siempre?
4. ¿Cuándo usarías inyección por constructor y cuándo por setter?

CLASE 7

Ecosistema Spring y Arquitectura en Capas

Model · Service · Repository

El ecosistema de Spring

Spring no es un único bloque monolítico — es una **familia modular de proyectos**. Cada módulo resuelve un desafío específico y se agrega solo si lo necesitas.

Módulo	Para qué sirve
Spring Core	El núcleo. Contenedor IoC, BeanFactory, ApplicationContext e inyección de dependencias.
Spring MVC	Capa de presentación. Implementa MVC para apps web y APIs REST con JSP, Thymeleaf, etc.
Spring Persistence	Simplifica el acceso a datos (JDBC, JPA/Hibernate, Spring Data).
Spring Security	Autenticación, autorización y protección contra vulnerabilidades web (CSRF, XSS, etc.).
Spring Cloud	Microservicios: descubrimiento de servicios, configuración centralizada, enrutamiento, etc.
Spring Boot	Cimientos. Auto-configuración y servidores embebidos para arrancar apps fácilmente.

La 'fontanería' (Plumbing Code)

En ingeniería de software, **plumbing** es todo el código repetitivo de soporte que no aporta valor directo al negocio pero es imprescindible para que la app funcione: crear objetos, inyectar dependencias, manejar transacciones, conectar con BD.

■ Spring se encarga de toda la fontanería. Tú te enfocas exclusivamente en la lógica de negocio. Tu código Java queda limpio (POJOs) y fácil de probar.

Arquitectura en capas: Model - Service - Repository

Para garantizar la **separación de responsabilidades**, una app Spring se divide en capas especializadas. Cada capa tiene un propósito único y se comunica solo con su capa contigua a través de interfaces.

Capa	Componente	Responsabilidad	Regla de comunicación
Presentación	Servlet / Controller	Recibe peticiones HTTP, extrae datos y llama a la Capa de Servicio.	Se comunica con la Capa de Servicio.
Servicio (Business)	Service	Reglas de negocio, validaciones, lógica de negocio.	Se comunica con el Repositorio mediante interfaces.
Repositorio (Data)	Repository / DAO	Gestiona acceso a datos (CRUD).	Manipula y retorna objetos del Modelo.
Modelo (Domain)	Model / Entity	Representa las entidades del sistema.	Transaccionalmente entre todas las capas (POJOs).

■ La Capa de Presentación JAMÁS debe comunicarse directamente con el Repositorio o la Base de Datos. Todas las operaciones deben transitar por la Capa de Servicio para garantizar que se apliquen las reglas de negocio.

El flujo de una petición de punta a punta

```
1. Cliente HTTP → GET /estudiantes 2. Servlet → invoca estudianteService.listarEstudiantes() 3.
Service → invoca estudianteRepository.obtenerTodos() 4. Repository → lee registros de memoria/BD
5. Repository → retorna List<Estudiante> 6. Service → entrega lista al Servlet 7. Servlet →
responde HTML/JSON al cliente
```

■ Preguntas de repaso

1. ¿Qué es el 'plumbing code' y por qué Spring lo elimina?
2. ¿Por qué la Capa de Presentación no debe comunicarse directamente con el Repositorio?
3. ¿Qué módulo de Spring usarías si quieres proteger tu app con login y contraseñas?
4. ¿Qué diferencia hay entre Spring Boot y Spring Core?

CLASE 8

Contenedor IoC y Beans con XML

BeanFactory · ApplicationContext · applicationContext.xml

¿Qué es un Spring Bean?

Un **Spring Bean** es cualquier objeto Java cuya instanciación, ensamblado y ciclo de vida son gestionados por el Contenedor IoC. Son POJOs puros — no necesitan extender ninguna clase especial de Spring.

Contenedor	Interfaz Java	Características	Cuándo usar
BeanFactory	org.springframework.beans.factory.BeanFactory	Es el más básico. Eager loading (crea beans antes de pedirlos)	Solo en aplicaciones con memoria muy limitada
ApplicationContext	org.springframework.context.ApplicationContext	Extiende a BeanFactory. Eager loading por defecto. Siempre se inicializa al iniciar la aplicación.	Siempre se usa en la mayoría de aplicaciones.

El applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?> <beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="..."> <!-- 1. Bean del
Repositorio --> <bean id="estudianteRepositoryBean"
class="com.example.repository.EstudianteRepositoryInMemory" /> <!-- 2. Inyección por Constructor
--> <bean id="estudianteServiceBean" class="com.example.service.EstudianteServiceImpl">
<constructor-arg ref="estudianteRepositoryBean" /> </bean> <!-- 3. Inyección por Setter --> <bean
id="estudianteServiceSetterBean" class="com.example.service.EstudianteServiceSetterImpl">
<property name="estudianteRepository" ref="estudianteRepositoryBean" /> </bean> </beans>
```

Atributo	Qué hace
id	Identificador único del bean en el contenedor. Se usa en <code>getBean('id')</code> .
class	Nombre completamente cualificado de la clase a instanciar.
constructor-arg ref	Inyección por constructor — pasa el bean referenciado como argumento de constructor.
property name ref	Inyección por setter — llama al método <code>setNombre()</code> con el bean referenciado.

Tipo	Etiqueta XML	Cuándo usarlo
Constructor	<code><constructor-arg ref='...'></code>	Dependencias OBLIGATORIAS — el objeto no puede funcionar sin él.
Setter	<code><property name='...' ref='...'></code>	Dependencias OPCIONALES — el objeto puede funcionar sin él.

Ejecución standalone con ClassPathXmlApplicationContext

```
ClassPathXmlApplicationContext context = new
ClassPathXmlApplicationContext("applicationContext.xml"); // Spring ya: 1) leyó el XML, 2)
instanció los beans, 3) inyectó dependencias
EstudianteService servicio = (EstudianteService)
context.getBean("estudianteServiceBean");
servicio.listarEstudiantes().forEach(System.out::println); context.close(); // dispara los
destroy-method
```

■ Preguntas de repaso

1. ¿Qué diferencia hay entre lazy loading (BeanFactory) y eager loading (ApplicationContext)?
2. ¿Qué hace el atributo 'id' en un y para qué se usa después?
3. ¿Qué hace Spring internamente cuando lees el applicationContext.xml con ClassPathXmlApplicationContext?
4. ¿Por qué la inyección por constructor es preferida para dependencias obligatorias?

CLASE 9

Scopes y Ciclo de Vida del Bean

Singleton · Prototype · init-method · destroy-method

Alcance de los Beans (Bean Scopes)

El **scope** especifica cuántas instancias de un bean creará el contenedor IoC y cómo se compartirán.

Scope	Instancias	Comportamiento	Uso ideal
singleton (defecto)	Una sola por ApplicationContext	Spring la guarda en caché y la reutiliza siempre	Servicios y Repositorios sin estado mutable
prototype	Una nueva por cada consulta	Spring crea un objeto nuevo cada vez que se solicita	Objetos con estado mutable como carritos de compra, buques, etc.

```
<!-- Singleton (por defecto, mismo resultado sin el atributo) --> <bean
id="estudianteServiceSingleton" class="com.example.service.EstudianteServiceImpl"
scope="singleton"> <constructor-arg ref="estudianteRepositoryBean" /> </bean> <!-- Prototype -->
<bean id="carritoCompras" class="com.example.model.CarritoCompras" scope="prototype" />
```

■ Spring gestiona el ciclo de vida de los beans singleton (incluyendo su destrucción). Para los beans prototype, Spring los crea e inyecta, pero NO gestiona su destrucción — esa responsabilidad recae en el Garbage Collector.

Ciclo de vida del Bean

1. Instanciación → Spring invoca el constructor Java 2. Inyección DI → Spring asigna las dependencias (ref) 3. init-method → Se ejecuta el método de inicialización 4. Bean listo → Disponible para la aplicación ... (atiende peticiones) ... 5. destroy-method → Se ejecuta al abandonar el contenedor

```
<!-- Configuración en XML --> <bean id="repoLifecycle"
class="com.example.repository.EstudianteRepositoryWithLifecycle" init-method="iniciarRepositorio"
destroy-method="limpiarRecursos" /> // En la clase Java: public void iniciarRepositorio() {
System.out.println("Inicializando conexión y cargando datos..."); } public void limpiarRecursos()
{ System.out.println("Cerrando conexiones y liberando memoria..."); }
```

■ init-method es ideal para abrir conexiones o inicializar cachés. destroy-method es ideal para cerrar archivos o liberar conexiones. Ambos deben ser métodos públicos sin argumentos.

■ Preguntas de repaso

1. ¿Por qué usar scope='prototype' para un carrito de compras en una app multihilo?
2. ¿Qué pasa si dos peticiones simultáneas usan el mismo bean singleton que tiene un atributo de clase mutable?
3. ¿Qué hace context.close() en relación con el ciclo de vida de los beans?
4. Un bean prototype tiene un destroy-method. ¿Spring lo ejecuta cuando el bean ya no se usa? Justifica.

CLASE 10

Integrando Spring con Servlets

El anti-patrón y la solución correcta con ContextLoaderListener

El anti-patrón — recrear el contexto en cada petición

```
@WebServlet("/estudiantes-error") public class EstudianteServletError extends HttpServlet {  
    @Override protected void doGet(HttpServletRequest req, HttpServletResponse resp) { // ANTI-PATRÓN:  
        crea un nuevo contexto en CADA petición HTTP ApplicationContext context = new  
        ClassPathXmlApplicationContext("applicationContext.xml"); EstudianteService servicio =  
        (EstudianteService) context.getBean("estudianteServiceBean"); } }
```

Consecuencia	Detalle
Destrucción del rendimiento	Lee y parsea el XML desde disco en cada clic HTTP
Pérdida del scope Singleton	Se crean copias aisladas del contenedor por petición — los singletons dejan de serlo
Fugas de memoria	El Garbage Collector colapsa limpiando múltiples contenedores huérfanos

La solución — ContextLoaderListener

El ApplicationContext debe instanciarse **una sola vez** cuando Tomcat arranca, guardarse en el ServletContext (memoria compartida de la WebApp), y destruirse cuando la app se apaga.

```
<!-- web.xml --> <web-app ...> <!-- 1. Ubicación del XML de Spring --> <context-param>  
    <param-name>contextConfigLocation</param-name>  
    <param-value>classpath:applicationContext.xml</param-value> </context-param> <!-- 2. Listener que  
    inicializa el contexto al arrancar Tomcat --> <listener> <listener-class>  
    org.springframework.web.context.ContextLoaderListener </listener-class> </listener> </web-app>
```

Y en el Servlet se obtiene el contexto ya existente en el `init()`:

```
@WebServlet("/estudiantes") public class EstudianteServlet extends HttpServlet { private  
    EstudianteService estudianteService; @Override public void init() throws ServletException { //  
    Obtiene el contexto YA EXISTENTE del ServletContext WebApplicationContext context =  
    WebApplicationContextUtils.getRequiredWebApplicationContext(getServletContext());  
    this.estudianteService = (EstudianteService) context.getBean("estudianteServiceBean"); } @Override  
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) { List<Estudiante> lista =  
    this.estudianteService.listarEstudiantes(); // ... construir respuesta HTML } }
```

- El XML se lee una sola vez al arrancar Tomcat. Todas las peticiones HTTP comparten la misma instancia del servicio (singleton). Sin fugas de memoria. El ciclo de vida del contenedor está sincronizado con el de Tomcat.

Alternativa — contenedor estático (enfoque de Domiciano)

Una segunda forma válida para integrar Spring sin usar el ContextLoaderListener es crear un contenedor estático que se inicializa una sola vez:

```
public class Application { private static final ApplicationContext context = new
ClassPathXmlApplicationContext("applicationContext.xml"); public static ApplicationContext
getContext() { return context; } } // En el Servlet: public void init() { messageRepository =
(MessageRepository) Application.getContext().getBean("messageRepository"); }
```

■ Preguntas de repaso

1. ¿Por qué crear el ApplicationContext dentro de doGet() es un anti-patrón? Menciona las tres consecuencias.
2. ¿Qué hace el ContextLoaderListener y cuándo se ejecuta?
3. ¿Por qué se obtiene el bean en init() y no en doGet()?
4. ¿Cuál es la diferencia entre el ServletContext y el ApplicationContext de Spring?

CLASE 11

De XML a Anotaciones

@Component · @Service · @Repository · @Autowired

¿Por qué migrar de XML a anotaciones?

Mantener archivos XML gigantescos en proyectos modernos se vuelve complejo y propenso a errores tipográficos. Con anotaciones, la configuración vive directamente en el código — más concisa y con seguridad en tiempo de compilación.

Anotaciones Estereotipo

Con @ComponentScan, Spring explora automáticamente los paquetes Java en busca de clases marcadas con anotaciones estereotipo:

Anotación	Capa	Propósito
@Component	General/Utilitaria	Marcador genérico para cualquier bean administrado
@Repository	Persistencia	Maneja acceso a BD. Traduce excepciones de BD a D
@Service	Negocio	Contiene lógica de negocio, validaciones y orquestaci
@Controller / @RestController	Presentación / Web	Procesa peticiones HTTP en apps web (MVC o API R

```
@Repository public class EstudianteRepositoryImpl implements EstudianteRepository { public String findNombreEstudiante() { return "Juan Pérez"; } } @Service public class EstudianteServiceImpl implements EstudianteService { private final EstudianteRepository estudianteRepository; @Autowired // inyección automática por constructor public EstudianteServiceImpl(EstudianteRepository estudianteRepository) { this.estudianteRepository = estudianteRepository; } }
```

■ La buena práctica es inyectar por constructor (no por campo) y declarar el atributo como final. Esto facilita las pruebas unitarias sin necesidad de levantar el contexto de Spring.

Ciclo de vida con @PostConstruct y @PreDestroy

```
@Service public class ConexionService { public ConexionService() { System.out.println("1. Constructor: objeto instanciado"); } @PostConstruct // reemplaza init-method en XML public void init() { System.out.println("2. @PostConstruct: dependencias inyectadas"); } @PreDestroy // reemplaza destroy-method en XML public void cleanup() { System.out.println("3. @PreDestroy: contenedor cerrándose"); } }
```

Scope con @Scope

```
@Component @Scope("prototype") // nueva instancia por cada consulta/inyección public class CarritoCompras { }
```

■ Preguntas de repaso

1. ¿Qué diferencia hay entre `@Component` y `@Repository`? ¿Por qué `@Repository` hace algo extra?
2. ¿Qué reemplaza `@PostConstruct` en la configuración XML?
3. ¿Por qué es mejor inyectar por constructor que por campo (`@Autowired` directo en el atributo)?
4. Si no pones `@Scope` en un bean con anotación, ¿qué scope tiene por defecto?

CLASE 12

Java Config: @Configuration y @Bean

Reemplazando el XML por completo con clases Java

¿Cuándo usar Java Config?

Las anotaciones estereotipo (@Service, @Repository) funcionan para tus propias clases. Pero ¿qué pasa con clases de **librerías externas** que no puedes modificar? Para esos casos, Java Config permite registrar beans sin tocar el código fuente.

@Configuration y @Bean

```
@Configuration @ComponentScan(basePackages = "com.example") // escaneo automático
@PropertySource("classpath:application.properties") // carga propiedades public class AppConfig {
// @Bean registra el valor devuelto como bean en el contenedor @Bean public String
nombreAplicacion() { return "Sistema de Gestión Académica"; } // Bean de librería externa con ciclo
de vida y scope @Bean(initMethod = "init", destroyMethod = "cleanup") @Scope("singleton") public
ExternalLibraryService externalLibraryService() { return new ExternalLibraryService(); } }
```

Anotación	Qué hace
@Configuration	Indica que la clase contiene definiciones de beans — reemplaza el XML
@Bean	El valor devuelto por el método se registra como Spring Bean
@ComponentScan	Activa el escaneo automático de @Component, @Service, etc.
@PropertySource	Carga un archivo .properties en el entorno de Spring

AnnotationConfigApplicationContext

```
// Antes (XML): new ClassPathXmlApplicationContext("applicationContext.xml") // Ahora (Java
Config): AnnotationConfigApplicationContext context = new
AnnotationConfigApplicationContext(AppConfig.class); EstudianteService servicio =
context.getBean(EstudianteService.class); System.out.println(servicio.obtenerDetalle());
context.close();
```

Cuadro comparativo final

Aspecto	XML	Anotaciones (@Component)	Java Config (@Configuration)
Ubicación	Archivos XML externos	Directamente en las clases	Clases de configuración separadas
Escaneo	Manual (<bean class='...'>)	Automático con @ComponentScan	Manual (@Bean) o combinado
Inyección DI	<property> o <constructor-arg>	@Autowired	Parámetros en métodos @Bean
Casos de uso	Legacy / código antiguo	Componentes propios del proyecto	Librerías externas / beans complejos

■ Preguntas de repaso

1. ¿Cuál es la diferencia entre `@Configuration` y `@Component`?
2. ¿Por qué `@Bean` va sobre un método y no sobre una clase?
3. ¿Cuándo usarías Java Config (`@Configuration`) en lugar de anotaciones (`@Service`)?
4. ¿Qué reemplaza `AnnotationConfigApplicationContext` respecto a `ClassPathXmlApplicationContext`?

CLASE 13

Inyección Avanzada, @Value y SpEL

@Qualifier · @Primary · application.properties · Spring Expression Language

El problema de la ambigüedad — NoUniqueBeanDefinitionException

Si Spring encuentra **dos beans que implementan la misma interfaz** y alguien la inyecta con @Autowired, no sabe cuál elegir y lanza NoUniqueBeanDefinitionException.

Solución 1 — @Qualifier

```
@Service public class UsuarioController { private final NotificacionService notificacionService;
@Autowired public UsuarioController( @Qualifier("notificacionSmsServiceImpl") NotificacionService
svc) { this.notificacionService = svc; // Spring inyecta explícitamente el SMS } }
```

Solución 2 — @Primary

```
@Service @Primary // implementación preferida cuando hay ambigüedad sin @Qualifier public class
NotificacionEmailServiceImpl implements NotificacionService { ... }
```

■ ■ @Qualifier siempre tiene mayor prioridad que @Primary. Si pones ambos, @Qualifier gana.

@Value y application.properties

```
# src/main/resources/application.properties app.nombre=DocuKelo Academic Portal
app.max-usuarios-activos=150 notificacion.modo-prueba=true

@Service public class SistemaConfigService { @Value("${app.nombre}") private String nombreApp; //
"DocuKelo Academic Portal" @Value("${app.max-usuarios-activos}") private int maxUsuarios; // 150
(conversión automática) @Value("${notificacion.modo-prueba}") private boolean modoPrueba; // true
(conversión automática) @Value("${app.timeout:3000}") // valor por defecto si no existe private
int timeout; // 3000 }
```

SpEL — Spring Expression Language

Mientras \${...} extrae valores literales de properties, #{...} (SpEL) **evalúa expresiones** en tiempo de ejecución — aritmética, lógica, invocación de métodos, acceso a otros beans.

Expresión SpEL	Resultado
#{10 * 5 + 20}	70 (operación matemática)
#{sistemaConfigService.maxUsuarios > 100}	true (accede a otro bean)
#{'DocuKelo'.toUpperCase()}	'DOCUKELO' (método de String)
#{T(java.lang.Math).PI}	3.14159... (clase estática con T(...))
#{T(java.util.UUID).randomUUID().toString()}	UUID único en runtime
#{\${app.max-usuarios-activos} * 2}	300 (combina \${} y #{})

Sintaxis	Tipo	Cuándo usar
<code>\${propiedad}</code>	Lectura pasiva	Extraer un valor literal de <code>application.properties</code>
<code>#{expresion}</code>	Evaluación activa	Calcular algo, invocar métodos, referenciar otros beans

■ Preguntas de repaso

1. Tienes `EmailSenderImpl` y `SmsSenderImpl`, ambas implementando `NotificacionService`. ¿Cómo le dices a Spring cuál inyectar en un lugar específico?
2. ¿Qué diferencia hay entre `${...}` y `#{...}` en una anotación `@Value`?
3. ¿Para qué sirve el valor por defecto en `@Value("${app.timeout:3000}")`? ¿Cuándo se usa?
4. Escribe la expresión SpEL que multiplica el valor de la propiedad `app.max-usuarios-activos` por 2.

CLASE 14

Maven y Comandos de Linux

Ciclo de vida de Maven · Comandos esenciales para despliegue

Apache Maven — Gestión del ciclo de vida

Maven automatiza la compilación, pruebas y empaquetado de proyectos Java mediante un modelo de objeto de proyecto definido en pom.xml.

El ciclo de vida por defecto

validate → compile → test → package → verify → install → deploy

Fase	Qué hace
validate	Verifica que el pom.xml sea válido
compile	Compila .java → .class en target/classes
test	Ejecuta pruebas unitarias (JUnit, TestNG)
package	Empaqueta en .jar o .war en target/
verify	Verificaciones adicionales de calidad
install	Instala el paquete en el repositorio local (~/.m2)
deploy	Copia el paquete a un repositorio remoto (Nexus, Artifactory)

Comando	Qué ejecuta	Salida	Mejor para
mvn compile	validate → compile	target/classes	Verificación rápida de sintaxis
mvn clean package	clean → validate → compile → test → package	target/package (.war)	Crear el artefacto para producción
mvn clean install	clean → ... → install	~/.m2/repository	Proyectos multi-módulo con dependencias
mvn clean compile exec:java	clean → compile → ejecuta Main	Consola	Correr una app standalone Spring con XML

■ ■ 'clean' no es una fase del ciclo por defecto — es del ciclo de limpieza. Elimina el directorio target/ para garantizar un build desde cero. Siempre prefiere 'mvn clean package' sobre 'mvn package' para evitar artefactos viejos.

Comandos de Linux esenciales

Comando	Qué hace	Ejemplo
ls	Lista archivos y directorios	ls -la
cd	Cambia el directorio	cd carpeta1

pwd	Muestra el directorio actual	pwd
mkdir	Crea un directorio	mkdir nueva_carpeta
rm	Elimina archivos o directorios	rm -r carpeta1
cp	Copia archivos	cp archivo.txt copia.txt
mv	Mueve o renombra	mv archivo.txt carpeta/
cat	Muestra contenido de un archivo	cat archivo.txt
tail -f	Muestra líneas en tiempo real	tail -f logs/catalina.out
grep	Busca patrones en archivos	grep 'error' catalina.out
ssh	Conecta a servidor remoto	ssh usuario@192.168.1.10
scp	Copia archivos entre hosts	scp app.war usuario@ip:/ruta/
ss -tulnp	Ver todos los puertos abiertos	ss -tulnp
lsof -i :8080	Procesos en el puerto 8080	lsof -i :8080
kill -9 PID	Mata un proceso por su PID	kill -9 15432
chmod +x *.sh	Da permisos de ejecución	chmod +x *.sh
wget URL	Descarga un archivo de internet	wget https://...tomcat.zip
unzip archivo.zip	Descomprime un zip	unzip apache-tomcat.zip

■ Preguntas de repaso

1. ¿Qué diferencia hay entre mvn compile y mvn clean package?
2. ¿Por qué se recomienda siempre incluir 'clean' antes de 'package'?
3. ¿Para qué sirve mvn clean install y cuándo lo usarías?
4. ¿Qué comando usarías para ver qué proceso está usando el puerto 8080 y cómo lo matarías?

CLASE 15

Despliegue Remoto con SSH y SCP

Instalar Tomcat en Linux y subir un .war por red

El flujo completo de despliegue remoto

En el laboratorio de ICESI, varios PCs tienen Linux y Tomcat. El flujo para desplegar tu app en uno de esos equipos es:

```
1. Conectarse por SSH al PC remoto 2. Descargar Tomcat en el PC remoto 3. Darle permisos de ejecución a los scripts de Tomcat 4. Arrancar Tomcat 5. Desde tu máquina, subir el .war por SCP 6. Verificar que la app está corriendo
```

Paso a paso

1. Conectarse al PC remoto

```
ssh computacion@192.168.131.31 # o cualquier PC del 31 al 40
```

2. Descargar Tomcat

```
wget https://dlcdn.apache.org/tomcat/tomcat-11/v11.0.18/bin/apache-tomcat-11.0.18.zip
```

3. Descomprimir y limpiar

```
unzip apache-tomcat-11.0.18.zip rm apache-tomcat-11.0.18.zip
```

4. Dar permisos y arrancar

```
cd apache-tomcat-11.0.18/bin chmod +x *.sh JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64 ./startup.sh
```

5. Subir el .war desde tu máquina

```
scp ./miapp.war computacion@192.168.131.31:/home/computacion/apache-tomcat-11.0.18/webapps/
```

6. Verificar puertos y procesos

```
ss -tulnp # ver todos los puertos abiertos lsof -i :8080 # ver qué proceso usa el puerto 8080 tail -f logs/catalina.out # ver los logs en tiempo real # Si necesitas matar el proceso: kill -9 <PID>
```

Estructura de carpetas de Tomcat

Carpeta	Para qué sirve
bin/	Scripts de inicio y parada (startup.sh, shutdown.sh, catalina.bat)
conf/	Archivos de configuración (server.xml, web.xml)
lib/	Bibliotecas Java (.jar) necesarias para Tomcat
logs/	Registros del servidor (catalina.out)

webapps/	Aquí se despliegan las aplicaciones (.war o carpetas)
work/	Archivos temporales generados durante la ejecución
temp/	Archivos temporales del servidor

■ Una vez que copias el .war a webapps/, Tomcat lo detecta automáticamente y lo despliega. No necesitas reiniciar Tomcat. La app estará disponible en <http://IP:8080/nombre-del-war/>

■ Preguntas de repaso

1. ¿Qué hace SCP y en qué se diferencia de CP?
2. ¿Por qué es necesario hacer `chmod +x *.sh` antes de ejecutar `startup.sh`?
3. ¿Dónde debes colocar el archivo .war para que Tomcat lo despliegue automáticamente?
4. ¿Cómo verías los logs de Tomcat en tiempo real para detectar errores de arranque de tu app?

RESUMEN GENERAL

Todo lo que necesitas saber para el parcial

Bloque 1 — Redes y Servidores Web (Clases 1-4)

Concepto	Clave para recordar
TCP	Confiable, ordenado, 3-way handshake (SYN→SYN-ACK→ACK), cierre en
UDP	Rápido, sin conexión, sin garantías. Para streaming, juegos, DNS
Puerto	El número del apartamento. IP identifica la máquina, puerto identifica el pro
ServerSocket	Uno solo, escucha para siempre en un puerto
Socket conexión	Uno por cliente, nace al aceptar y muere al responder
CRLF	\r\n al final de cada línea HTTP. La línea vacía separa headers del body —
flush()	Empuja los bytes del buffer al socket. Sin él el cliente nunca recibe nada
while(true)	Bucle infinito del servidor multi-hilos. Se detiene con Ctrl+C
Condición de carrera	Buffer como atributo de clase = compartido entre hilos = corrupción

Bloque 2 — Tomcat y Servlets (Clase 5)

Concepto	Clave para recordar
Coyote Connector	Parsea HTTP y construye HttpServletRequest/Response — elimina el trab
Catalina Engine	Gestiona el pool de hilos y decide qué Servlet atiende cada petición
Ciclo de vida Servlet	init() una vez → service()/doGet()/doPost() por petición → destroy() una ve
Una instancia	Tomcat crea UN Servlet y lo reutiliza. Los hilos son los que se crean por pe
scope='provided'	Tomcat ya trae jakarta.servlet-api. No incluirla en el .war evita conflictos
sendRedirect()	Genera 302 Found + Location. El navegador hace automáticamente un GE

Bloque 3 — Spring Framework (Clases 6-13)

Concepto	Clave para recordar
SOLID	5 principios de diseño. Los más importantes para Spring: S (una responsab
DIP	Alto nivel no depende de bajo nivel. Ambos dependen de interfaces (abstra

IoC	El control de creación de objetos se delega al contenedor de Spring
Spring Bean	Objeto gestionado por Spring. Es un POJO puro — no extiende nada especial
ApplicationContext	Contenedor estándar. Eager loading + eventos + I18N. Siempre úsalo sobre el genérico
constructor-arg	Inyección por constructor en XML — para dependencias obligatorias
property	Inyección por setter en XML — para dependencias opcionales
scope singleton	Una instancia compartida. Defecto. Para Servicios y Repositorios stateless
scope prototype	Nueva instancia por consulta. Para objetos con estado (carritos, reportes)
init-method / destroy-method	Ciclo de vida del bean en XML
ContextLoaderListener	Crea el ApplicationContext UNA vez al arrancar Tomcat. Evita el anti-patrón
@Component/@Service/@Repository	Registran la clase como bean automáticamente con @ComponentScan
@Autowired	Inyección automática. Mejor por constructor con atributo final
@PostConstruct / @PreDestroy	Equivalen a init-method / destroy-method pero con anotaciones
@Configuration + @Bean	Java Config. Reemplaza el XML. Ideal para librerías externas
@Qualifier	Selección explícita cuando hay ambigüedad. Gana sobre @Primary
@Primary	Implementación preferida por defecto cuando hay ambigüedad
@Value("\${...}")	Inyecta valor literal de application.properties
SpEL #{...}	Evalúa expresiones en runtime: matemáticas, métodos de beans, clases especiales

Bloque 4 — Maven y Despliegue (Clases 14-15)

Concepto	Clave para recordar
mvn clean package	El comando más usado. Limpia + compila + prueba + empaqueta en .war/.jar
mvn clean install	Como package pero además instala en ~/.m2 para proyectos multi-módulo
mvn clean compile exec:java	Compila y ejecuta Main standalone de Spring con XML
webapps/	Carpeta de Tomcat donde se colocan los .war para despliegue automático
scp	Copia archivos entre máquinas de forma segura por SSH
lsof -i :8080	Ver qué proceso usa el puerto 8080
kill -9 PID	Matar un proceso Linux por su PID
tail -f catalina.out	Ver logs de Tomcat en tiempo real

¡Éxito en el parcial, Juliana! ■

